

```
# Linear search
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

data = [10, 20, 30, 40, 50]
print(linear_search(data, 30))
```

2

```
# Binary search (iteratif)
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

data = [10, 20, 30, 40, 50]
print(binary_search(data, 40))
```

3

```
# Binary Search (Rekursif)
def binary_search_recursive(arr, left, right, target):
    if left > right:
        return -1

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, mid + 1, right, target)
    else:
        return binary_search_recursive(arr, left, mid - 1, target)

data = [10, 20, 30, 40, 50]
print(binary_search_recursive(data, 0, len(data)-1, 20))
```

1

```
# Jump Search
import math

def jump_search(arr, target):
    n = len(arr)
    step = int(math.sqrt(n))
    prev = 0

    while arr[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))

    if prev >= n:
        return -1

    for i in range(prev, min(step, n)):
        if arr[i] == target:
            return i

    return -1

data = [10, 20, 30, 40, 50, 60, 70]
print(jump_search(data, 60))
```

5

```
# Interpolation Search
def interpolation_search(arr, target):
```

```
low = 0
high = len(arr) - 1

while low <= high and target >= arr[low] and target <= arr[high]:

    if low == high:
        if arr[low] == target:
            return low
        return -1

    pos = low + ((target - arr[low]) * (high - low) //
                (arr[high] - arr[low]))

    if arr[pos] == target:
        return pos

    elif arr[pos] < target:
        low = pos + 1

    else:
        high = pos - 1

return -1

data = [10, 20, 30, 40, 50]
print(interpolation_search(data, 50))
```

4